Vehicle Parking System Software Test Plan

Version 2.0 | 8 November 2026

Prepared by Vehicle Parking System Project Team

Application baseline: 0c883054fc84897bf303c3b766b60e0152fac1e9 with the measured vehicle-search batching refinement included in this delivery. Requirements, architecture, tests and the delivery register use the same canonical FR and NFR identifiers.

Contents

1 Introduction 2
2 Test Items 2
3 Features to be Tested 2
4 Features Not to be Tested 2
5 Test Approach 3
6 Environment 4
7 Schedule 4
8 Deliverables 4
9 Responsibilities 4
10 Risks and Controls 4
11 Assumptions and Dependencies 4
12 Suspension and Resumption 5
13 Test Cases and Traceability 5
14 Metrics and Reporting 6
15 Review and Acceptance 6

1 Introduction

This plan uses the IEEE 829 style test-plan sections used in the course. It defines test items, features, approach, environment, responsibilities, execution and acceptance evidence. The current SRS and SAD are the baseline. Automated outcomes describe the checks actually run and remain distinct from team/instructor acceptance.

2 Test Items

The test items are accounts, vehicles, facility, reservations, parking operations, pricing/billing, reports/exports, audit, notifications and shared errors/security. The catalogue links every case to SRS IDs and executable test modules. Test data is synthetic and contains no real payment or personal information.

3 Features to be Tested

Area	SRS coverage	Checks
Identity and access	FR-15, FR-16, FR-17, FR-18	Positive, invalid, ownership, boundary and state cases in accounts
Vehicle management	FR-01, FR-02, FR-11	Positive, invalid, ownership, boundary and state cases in vehicles
Facility and availability	FR-03, FR-04, FR-07, FR-10	Positive, invalid, ownership, boundary and state cases in parking_slots
Reservations	FR-19, FR-20, FR-21, FR-22	Positive, invalid, ownership, boundary and state cases in reservations
Parking operations	FR-05, FR-06, FR-08, FR-09, FR-12	Positive, invalid, ownership, boundary and state cases in operations
Pricing and settlement	FR-23, FR-24, FR-25, FR-26	Positive, invalid, ownership, boundary and state cases in billing
Reporting	FR-27, FR-28	Positive, invalid, ownership, boundary and state cases in reports
Shared services	FR-13, FR-14, FR-29, FR-30	Positive, invalid, ownership, boundary and state cases in common

NFR testing covers role/secret controls, atomic rollback, duplicate allocation, four-worker CRUD latency and synchronized 50-request races. Search/history measurements use 100000 vehicles and completed sessions, with exact and partial queries evaluated separately.

4 Features Not to be Tested

Real-money merchant processing, camera/barrier hardware, external identity services and framework/database internals are separate integration concerns. The suite verifies application behavior over

the configured database. Local HTTP measurements do not measure public-network or TLS proxy latency; deployment verification records those environments separately.

5 Test Approach

5.1 Security Validation

- Validate every exercised protected role/ownership boundary by direct API requests, including requests that bypass frontend navigation.
- Validate current-password re-entry, hashing, token expiry/revocation, public Customer registration and administrative account safeguards.
- Validate allowed/disallowed origins, CSRF and production HTTPS/Secure-cookie settings.
- Exercise malformed/injection-like input; verify error envelopes and audit data exclude secrets.
- Run Bandit 1.9.4 at high severity and the repository secret scan in CI; identify scope and tool versions in evidence.
- Map security outcomes to NFR-SEC-01 through NFR-SEC-07. Deployment certificate verification is a separate environment check.

5.2 Test levels and design techniques

Unit tests cover calculations, normalization and helpers. API/integration tests cover permissions, state transitions and persistence. PostgreSQL transaction tests use independent connections for real row-lock/constraint behavior. Frontend helper tests, lint and production build verify client logic and packaging. Equivalence partitions, boundary values, state transitions and fault injection select the important cases.

5.3 Performance and concurrency method

scripts/verify_acceptance.py creates a fresh uniquely named Django PostgreSQL test database, applies migrations and seeds 100000 vehicles plus 100000 completed sessions. It runs a real threaded WSGI HTTP server on loopback with DEBUG=False and real Bearer JWT authentication. Only the benchmark disables local HTTPS redirection. Five warm-ups precede 100 samples. CRUD uses four workers; search/history are measured serially. p95 uses nearest rank ceil(0.95*N). Each allocation/checkout race releases 50 client threads from a barrier. Timing includes response body read. Raw samples, status counts, runtime versions and integrity counts are retained in JSON. The command removes only its own test database.

5.4 Entry exit and regression rules

Entry requires reviewable code, reproducible dependencies, a dedicated test database, synthetic fixtures and agreed expected results. Exit requires executed scoped cases, valid response/status assertions, verified invariants, documented measurements and reviewed evidence. Fixes rerun affected checks; a final complete PostgreSQL regression validates integration. Team sign-off follows review of the evidence and the selected demonstration workflows.

6 Environment

The final verification report records exact Python/Django/PostgreSQL/Node versions, operating system, execution date and application baseline. Backend dependencies come from requirements-postgres.txt, frontend dependencies from npm ci. Tests run in isolated PostgreSQL databases; existing development data is unchanged. CI independently checks Ubuntu/Python 3.13/Node 22.

7 Schedule

Sprint 1 and Sprint 2 delivery are recorded in the synchronized board register. Integration verification and documentation consolidation were executed on 8 November 2026. Each new change reruns applicable checks before review. The final release baseline identifies a concrete application commit and its verification refinement.

8 Deliverables

SRS, SAD, this plan, the canonical test catalogue, RTM, final verification report, performance JSON, regression execution JSON and the team delivery register are maintained in docs. Editable UML sources accompany SVG and PNG diagrams. All generated documents retain team attribution.

9 Responsibilities

Member 1 owns identity/vehicles; Member 2 owns facility/reservations except US-11/12; Member 3 owns operations/billing; Member 4 owns reporting/audit/notifications/shared quality and reassigned US-11/12. Story authors retain their original PR attribution. All owners maintain their tests and participate in review; the team reviews release readiness.

10 Risks and Controls

Database-specific behavior is tested on PostgreSQL. Synthetic fixtures isolate tests from application data. Barrier races measure actual contention. Warm-ups and raw timings make performance reproducible. Source/configuration checks complement endpoint checks. Recorded measurements apply to their documented local profile rather than an unspecified hosting environment.

11 Assumptions and Dependencies

The configured PostgreSQL test account can create temporary databases. The application has no required external merchant or hardware dependency for its software workflows. Local benchmark CPU/database activity should be otherwise idle during measured samples. Test secrets are generated or environment-supplied and excluded from reports.

12 Suspension and Resumption

Suspend an affected run if the server/database becomes unavailable, fixtures are invalid or an invariant cannot be evaluated. Record the condition and affected case. Resume after the environment is restored and rerun that complete measurement group with a fresh dataset. Only completed valid runs enter the final report.

13 Test Cases and Traceability

The canonical catalogue in TEST_CASES.md provides stable case IDs, preconditions, procedures, expected outcomes, sources and execution status. It replaces ambiguous older representative IDs; historical IDs remain contextual references in archived snapshots. The RTM maps requirements directly to these canonical cases.

Case	Requirements	Scenario
TC-AUTH-01	FR-15	Submit login then access protected identity and logout
TC-AUTH-02	FR-18 NFR-SEC-05	Update profile; change password; retry old tokens
TC-SEC-01	FR-16 FR-17 NFR-SEC-01 NFR-SEC-02	Exercise role matrix and administrative account changes
TC-VEH-01	FR-01 FR-02 FR-13	Register vehicle then read and update its profile
TC-VEH-02	FR-13 FR-14	Submit valid malformed and concurrent duplicate registrations
TC-VEH-03	FR-11	Search exact and partial plates; serialize 25 search results
TC-SLOT-01	FR-03 FR-04	Create update transition and remove eligible slot
TC-SLOT-02	FR-07 NFR-SAFE-01	Attempt destructive changes and conflicting allocation
TC-SLOT-03	FR-10	Read filtered availability and compare permitted fields
TC-RES-01	FR-19	Create reservation with valid and invalid ownership/window
TC-RES-02	FR-20 FR-21	Modify cancel and overlap reservations including buffer boundaries
TC-RES-03	FR-22	Check in with reservation code and exercise invalid linkage
TC-OPS-01	FR-05 FR-06 FR-25	Check in; inspect token and snapshots; attempt duplicate entry
TC-OPS-02	FR-08 FR-09 FR-24	Check out at grace and hourly boundaries; repeat checkout

Case	Requirements	Scenario
TC-OPS-03	NFR-SAFE-03	Raise a persistence/release exception before transaction completion
TC-OPS-04	FR-12	Read paginated/filtered history after changing current labels
TC-BILL-01	FR-23 FR-24	Configure rate and calculate grace/hour/multiplier cases
TC-BILL-02	FR-25 FR-26	Generate receipt twice; settle or waive; retry invalid transitions
TC-REP-01	FR-27 FR-28	Query dashboard occupancy and collected revenue by interval
TC-REP-02	FR-28	Export CSV/PDF; inspect filters headers snapshots and formula protection
TC-AUD-01	FR-29 NFR-SEC-07	Trigger events; read as Admin; attempt update/delete
TC-NOT-01	FR-30	Produce events; inspect own inbox; dismiss; simulate dispatch error
TC-VAL-01	FR-13 FR-14	Exercise exception handler and frontend error helper
TC-SEC-02	NFR-SEC-04	Exercise CORS CSRF HTTPS redirect and cookie configuration
TC-SEC-03	NFR-SEC-06	Exercise safe query input and run Bandit high-severity scan

14 Metrics and Reporting

Report executed/passed/failed/skipped counts independently, database engine, commands, baseline and date. Do not add overlapping suite counts to imply unique coverage. For latency report sample size, concurrency, p50/p95/max and response counts. For races report exactly-one-success and database invariants. Bandit results describe high-severity scope; no scan claims universal absence of vulnerabilities.

15 Review and Acceptance

The plan is prepared for team review with executed evidence. Repository PR review records code/document review; course acceptance is recorded by the evaluating team/instructor. Historical merge/review records are preserved as evidence rather than reconstructed approvals.